

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

QuizNet+ – Hệ thống Quiz Game đa người chơi thời gian thực

1. Giới thiệu

1.1 Mục đích

QuizNet+ là hệ thống trò chơi đố vui (Quiz Game) nhiều người chơi hoạt động theo mô hình Client-Server và giao tiếp thời gian thực bằng WebSocket.

Hệ thống được xây dựng nhằm áp dụng các kỹ thuật quan trọng trong môn Computer Networking như:

- WebSocket realtime communication
- Multi-client concurrent handling
- Async/Await processing
- Realtime synchronization
- Multiplayer room management
- In-memory runtime data management
- Broadcast messaging
- Thread-safe collections
- Timer synchronization
- Session lifecycle management

Người chơi có thể:

- Nhập nickname để tham gia hệ thống
- Tạo phòng chơi
- Tham gia phòng

- Chọn bộ câu hỏi
- Trả lời câu hỏi realtime
- Chat với người chơi khác
- Xem bảng điểm realtime
- Tính điểm dựa trên tốc độ trả lời

1.2 Phạm vi sản phẩm

Trong phạm vi

Hệ thống hỗ trợ:

- Người chơi nhập nickname để tham gia hệ thống.
- Lobby realtime.
- Tạo và tham gia room.
- Gameplay realtime nhiều người chơi.
- Chat realtime trong room.
- Tính điểm theo thời gian trả lời.
- Đồng bộ trạng thái realtime.
- Load Question Set từ JSON files.
- Quản lý nhiều client đồng thời.
- Heartbeat connection checking.
- Host migration.
- Quản lý dữ liệu bằng in-memory collections.

Ngoài phạm vi

Hệ thống không hỗ trợ:

- Database persistent storage.
- Login/Register account.
- Voice chat.

- Video streaming.
- Mobile application.
- Payment system.
- Public Internet deployment.

1.3 Định nghĩa và viết tắt

Thuật ngữ	Ý nghĩa
SRS	Software Requirement Specification
BE	Backend
FE	Frontend
Host	Người tạo phòng
Player	Người tham gia
Lobby	Sảnh chờ
Room	Phòng chơi
Question set	Bộ câu hỏi
Runtime Data	Dữ liệu lưu tạm trong memory
Web socket	Giao thức giao tiếp realtime hai chiều
Broadcast	Gửi dữ liệu tới nhiều client cùng lúc
Session	Phiên kết nối của player

2. Tổng quan hệ thống

2.1 Kiến trúc hệ thống

Hệ thống sử dụng mô hình Client-Server:

Frontend Client

↕ WebSocket

ASP.NET Core Server

↕

JSON Question Files + Runtime Memory

Backend (BE)

Backend được phát triển bằng:

- C#
- ASP.NET Core (.NET 8)
- Visual Studio Code (VS Code)

Backend chịu trách nhiệm:

- Quản lý WebSocket connections.
- Quản lý player sessions.
- Quản lý lobby.
- Quản lý rooms.
- Gameplay logic.
- Timer synchronization.
- Score calculation.
- Chat broadcasting.

- Concurrent client handling.
- Heartbeat checking.
- Host migration.
- Load Question Set từ JSON files.

Frontend (FE)

Frontend được phát triển bằng:

- HTML5
- CSS3
- JavaScript
- Visual Studio Code (VS Code)

Frontend chịu trách nhiệm:

- Hiển thị giao diện.
- Kết nối WebSocket tới server.
- Gửi và nhận dữ liệu realtime.
- Hiển thị gameplay.
- Hiển thị timer.
- Hiển thị scoreboard.
- Hiển thị trạng thái room realtime.

Runtime Data Storage

Hệ thống không sử dụng database.

Dữ liệu runtime được lưu bằng:

- Dictionary
- ConcurrentDictionary
- List

Question Sets được lưu bằng:

- JSON files phía server

Dữ liệu runtime sẽ bị xóa khi server shutdown.

2.2 Development Environment

Thành phần	Công nghệ sử dụng
IDE	Visual Studio Code
Backend Language	C#
Framework	ASP.NET Core (.NET 8)
Frontend	HTML/CSS/JavaScript
Realtime Communication	WebSocket
Runtime Storage	ConcurrentDictionary / List
Concurrency	Async/Await
Version Control	Git/GitHub

Operating System	Windows 11
------------------	------------

2.3 Giao tiếp hệ thống

HTTP

Backend phục vụ frontend bằng HTTP Static File Server.

Ví dụ:

`http://<server-ip>:8888`

WebSocket

Frontend kết nối realtime tới backend bằng WebSocket.

Endpoint:

`ws://<server-ip>:8888/ws`

WebSocket được sử dụng cho:

- Gameplay realtime
- Room synchronization
- Chat realtime
- Scoreboard updates
- Timer synchronization
- Lobby updates
- Heartbeat connection checking

2.4 Concurrent Client Handling

Server hỗ trợ nhiều client đồng thời bằng:

- Async/Await
- Task-based concurrency
- ConcurrentDictionary
- Non-blocking WebSocket handling

Server không tạo blocking thread riêng cho từng client nhằm giảm tải nguyên hệ thống.

2.5 Gameplay State Machine

WAITING

→ START

→ QUESTION

→ ANSWERING

→ RESULT

→ NEXT

→ GAME_OVER

2.6 Session Lifecycle

CONNECT

→ ENTER_NICKNAME

→ JOIN_LOBBY

→ CREATE_ROOM / JOIN_ROOM

→ GAMEPLAY

→ DISCONNECT

2.7 Broadcast Mechanism

Server broadcast dữ liệu realtime tới các client thông qua WebSocket connections.

Các loại broadcast gồm:

- Lobby updates
- Room updates
- Chat messages
- Question broadcasts
- Scoreboard updates
- Player disconnect notifications

Gameplay messages chỉ được broadcast tới các client trong cùng room.

2.8 Cấu trúc dự án

QuizNet/

|— server/

| |— Managers/

| |— Models/

| |— Services/

| |— WebSocket/

| |— Utils/

| |— Questions/

| | └─ science.json

| | └─ history.json

| | └─ anime.json

| └─ wwwroot/

| | └─ css/

| | └─ js/

| | └─ assets/

| └─ Program.cs

|

└─ [README.md](#)

3. Yêu cầu chức năng

3.1 Player Session Management

FR1.1 – Temporary Nickname

Người chơi nhập nickname trước khi tham gia hệ thống.

Nickname chỉ tồn tại trong runtime hiện tại.

FR1.2 – Nickname Validation

Server kiểm tra nickname:

- Không được trống.
- Không vượt quá độ dài quy định.
- Không được trùng với player đang online.

FR1.3 – Active Session

Sau khi nhập nickname:

- Server tạo temporary session cho player.
- Player được chuyển tới lobby.

FR1.4 – Online Player Management

Server quản lý:

- Danh sách player online.
- Active sessions.
- Current rooms.

FR1.5 – Player Join/Leave Notification

Server broadcast thông báo khi:

- Player tham gia hệ thống.

- Player disconnect.
- Player rời room.

Ví dụ:

[System] Thành đã tham gia hệ thống

FR1.6 – Runtime Session Limitation

Do hệ thống không sử dụng database:

- Nickname
- Sessions
- Rooms
- Gameplay states

sẽ bị xóa khi server shutdown.

3.2 Lobby System

FR2.1 – Lobby Screen

Sau khi nhập nickname thành công, player được chuyển tới Lobby.

Lobby hiển thị:

- Danh sách room.
- Danh sách Question Sets.
- Danh sách player online.

FR2.2 – Lobby Actions

Player có thể:

- Tạo room.
- Join room.

- Chọn Question Set khi tạo room.

FR2.3 – Room List

Lobby hiển thị:

- Room name
- Số lượng player
- Room status

FR2.4 – Realtime Lobby Update

Server broadcast realtime khi:

- Room được tạo.
- Room bị xóa.
- Player join room.
- Player leave room.

3.3 Question Set Management

FR3.1 – Question Set Files

Question Sets được lưu dưới dạng JSON files phía server.

Ví dụ:

science.json

history.json

anime.json

FR3.2 – Load Question Set

Host chọn Question Set khi tạo room.

Server load Question Set từ JSON file vào memory.

FR3.3 – Question Structure

Mỗi câu hỏi gồm:

```
{  
  
  "content": "Question?",  
  
  "A": "...",  
  
  "B": "...",  
  
  "C": "...",  
  
  "D": "...",  
  
  "correct": "A"  
  
}
```

FR3.4 – Runtime Question Storage

Server sử dụng:

- ConcurrentDictionary
- List<Question>

để quản lý question data trong runtime.

3.4 Room Management

FR4.1 – Create Room

Host tạo room bằng cách chọn:

- Room name

- Question Set
- Number of questions (optional)

FR4.2 – Join Room

Player có thể join room từ Lobby.

FR4.3 – Room Synchronization

Server realtime synchronize:

- Danh sách player.
- Room status.
- Current gameplay state.

FR4.4 – Host Permission

Host có quyền:

- Start game.
- Chọn Question Set.
- Quản lý room trước khi game bắt đầu.

FR4.5 – Player Permission

Player có thể:

- Chat
- Trả lời câu hỏi

Player không thể:

- Start game
- Chọn Question Set
- Thay đổi room settings

FR4.6 – Host Migration

Nếu host disconnect trong WAITING state:

- Server tự động chuyển quyền host cho player khác trong room.

FR4.7 – Room Cleanup

Nếu room không còn player:

- Server tự động xóa room khỏi runtime memory.

3.5 Gameplay System

FR5.1 – Start Game

Host nhấn Start Game để bắt đầu.

Server load Question Set từ memory.

FR5.2 – Question Broadcast

Server gửi câu hỏi realtime tới toàn bộ player trong room thông qua WebSocket.

FR5.3 – Timer System

Mỗi câu hỏi có:

- Timer mặc định 20 giây.

Server quản lý timer bằng:

- Async Task
- CancellationToken

Frontend hiển thị countdown realtime.

FR5.4 – Answer Submission

Tất cả player có thể trả lời đồng thời.

Mỗi player chỉ được gửi 1 answer cho mỗi câu hỏi.

FR5.5 – Timestamp Validation

Server sử dụng timestamp phía server để:

- Đảm bảo tính công bằng.
- Chống fake timestamp từ client.

FR5.6 – Scoring System

Điểm được tính dựa trên:

- Độ đúng.
- Thời gian trả lời.

Công thức:

$$\text{Score} = \text{BaseScore} + \text{Bonus}$$

$$\text{Bonus} = (T - t) \times K$$

Trong đó:

- BaseScore = 100
- T = thời gian tối đa
- t = thời gian đã trôi qua
- K = hệ số điểm thưởng

FR5.7 – Result Broadcast

Sau mỗi câu hỏi, server broadcast:

- Đáp án đúng.
- Điểm từng player.
- Bảng xếp hạng tạm thời.

FR5.8 – Game Over

Sau câu cuối:

Server gửi:

- Final scoreboard
- Winner information

3.6 Chat System

FR6.1 – Room Chat

Player có thể chat trong room.

FR6.2 – Realtime Broadcast

Server broadcast chat message realtime tới toàn bộ player trong room.

FR6.3 – Chat Validation

Server kiểm tra:

- Message length
- Invalid content
- Dangerous characters

để tránh spam hoặc XSS.

FR6.4 – Rate Limiting

Server giới hạn số lượng chat messages trong khoảng thời gian nhất định để tránh spam.

3.7 Concurrent Client Handling

FR7.1 – Multi-client Support

Server hỗ trợ nhiều client đồng thời.

FR7.2 – Async Processing

Server sử dụng:

- Async/Await
- Task-based concurrency

để xử lý:

- WebSocket connections
- Room handling
- Timer
- Broadcast

FR7.3 – Concurrent Rooms

Nhiều room có thể hoạt động đồng thời.

FR7.4 – Graceful Disconnect

Server xử lý client disconnect mà không crash hệ thống.

FR7.5 – Thread-safe Collections

Server sử dụng:

- ConcurrentDictionary

để tránh race condition khi nhiều client truy cập cùng lúc.

3.8 Heartbeat System

FR8.1 – Heartbeat Message

Server gửi heartbeat định kỳ tới client để kiểm tra kết nối.

FR8.2 – Timeout Detection

Nếu client không phản hồi heartbeat trong khoảng thời gian quy định:

- Server đóng session.
- Cleanup resources.
- Broadcast disconnect event.

3.9 WebSocket Protocol

Message Structure

Tất cả dữ liệu giữa FE ↔ BE sử dụng JSON:

```
{  
  
  "type": "COMMAND",  
  
  "data": {},  
  
  "timestamp": 123456,  
  
  "requestId": "uuid"  
}
```

Main Message Types

Type	Chức năng
JOIN_LOBBY	Vào lobby
CREATE_ROOM	Tạo room

JOIN_ROOM	Tham gia phòng
ROOM_UPDATED	Đồng bộ room
START_GAME	Bắt đầu game
QUESTION	Gửi câu hỏi
ANSWER	Gửi câu trả lời
CHAT	Nhắn tin
RESULT	Kết quả
SCOREBOARD	Bảng điểm
GAME_OVER	Kết thúc game
ERROR	Báo lỗi
HEARTBEAT	Kiểm tra kết nối

Message Flow Example

Client → JOIN_ROOM

Server → ROOM_UPDATED

Server → PLAYER_JOINED

4. Yêu cầu phi chức năng

4.1 Hiệu năng

Hệ thống phải:

- Hỗ trợ tối thiểu 50 client đồng thời.
- Broadcast realtime < 100ms trong LAN.
- Timer ổn định và chính xác.

4.2 Độ tin cậy

Backend phải:

- Không crash khi client disconnect.
- Tự cleanup session lỗi.
- Có heartbeat mỗi 10 giây.
- Tự động cleanup room rỗng.

4.3 Bảo mật

Hệ thống phải:

- Validate input.
- Sanitize chat message.
- Kiểm tra dữ liệu WebSocket hợp lệ.

4.4 Maintainability

Hệ thống phải:

- Dễ mở rộng module.
- Dễ bảo trì bằng VS Code.

- Tách biệt rõ frontend/backend/runtime-data layer.

4.5 Khả năng mở rộng

Hệ thống hỗ trợ:

- Nhiều room hoạt động đồng thời.
- Thêm game mode mới.
- Thêm Question Set mới.

4.6 Usability

Frontend phải:

- Responsive.
- Timer rõ ràng.
- Gameplay dễ thao tác.

5. Giao diện hệ thống

5.1 Frontend Screens

Frontend gồm các màn hình:

- Nickname Input
- Lobby
- Room
- Gameplay
- Result

5.2 Backend Console

Backend hiển thị:

- Connection logs
- Room logs
- Error logs
- Disconnet logs

5.3 Network

- HTTP Port: 8888
- WebSocket Endpoint: [/ws](#)

6. Runtime Data Management

6.1 In-Memory Storage

Server sử dụng:

- ConcurrentDictionary
- Dictionary
- List

để lưu dữ liệu runtime.

6.2 JSON Question Files

Question Sets được lưu dưới dạng JSON files phía server.

Server load các file này khi: khởi động hệ thống hoặc host tạo room.

6.3 Runtime Objects

Server lưu:

- Active players
- Active sessions
- Active rooms
- Current game states
- Loaded question sets

6.4 Runtime Data Limitation

Do không sử dụng database:

- Dữ liệu sẽ mất khi server shutdown.

- Không hỗ trợ persistent storage.

7. Thiết kế hệ thống

7.1 Backend Modules

Module	Chức năng
SessionManager	Quản lý player sessions
ConnectionManager	WebSocket handling
LobbyManager	Lobby handling
RoomManager	Room management
QuestionManager	Load JSON question files
GameManager	Gameplay logic
ChatManager	Chat broadcasting

7.2 Main Classes

- Player
- PlayerSession
- Room
- GameSession
- Question
- QuestionSet
- Message

8. Use Cases

UC1 – Join Lobby

1. Player nhập nickname
2. Server tạo session
3. Player được chuyển tới Lobby

UC2 – Create Room

1. Host chọn Question Set
2. Host tạo room
3. Player join room

UC3 – Gameplay

1. Host start game
2. Server load Question Set
3. Server gửi câu hỏi
4. Player trả lời
5. Server tính điểm
6. Hiển thị kết quả

UC4 – Disconnect Handling

1. Client disconnect
2. Server detect timeout
3. Remove session
4. Update room state
5. Broadcast disconnect event

9. Phụ lục

Bao gồm:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- State Diagram
- Deployment Diagram
- WebSocket Message Flow Diagram